ADANI UNIVERSITY

Faculty of Engineering Sciences and Technology (FEST)

Department of Computer Science and Engineering (AI-ML)

PROJECT-BASED LEARNING (PBL) & RESEARCH IMPLEMENTATION

Course: ECSCT24301 – Foundation of AI

PROJECT / RESEARCH PAPER PROPOSAL

Applied AI Project Track

Academic Year	2026–27
Project Team Member 1	KRUSHAL PATEL - 2410111714
Project Team Member 2	JAMBUKIYA DAKSHIT - 2410111044
Project Team Member 3	PARMAR PRINCE - 2410111078

1. Project Title & Track

Project Title: Lost Item Recovery Agent: An Intelligent Search Agent for Recovering Misplaced Objects in a Dynamic Environment

Selected Track: Applied AI Project Track

2. Problem Statement & Motivation

Students frequently misplace items such as mobile phones, wallets, ID cards, calculators, earbuds, or chargers within a college or campus environment. Usually, the person remembers only the last known location and a sequence of places visited afterward. Searching every location randomly can consume unnecessary time and effort.

The proposed system models the college/building environment as a state-space and develops an intelligent search agent that recommends an efficient order in which locations should be searched. The project will compare an uninformed baseline search with informed search using heuristics. The objective is to reduce search cost, execution time, and unnecessary state exploration.

3. Agent & Environment Characterization (PEAS)

PEAS Component	Proposed Specification
Performance Measure	Search cost, total distance, execution time, number of nodes expanded, and successful recovery.
Environment	A simulated college/building/campus map containing classrooms, laboratories, corridors, library, canteen,

	and other locations.
Actuators	Move to a location, search a location, update the current state, and generate/recalculate a search route.
Sensors	Last-known location, visited-location history, map connectivity, location information, and search results.

4. Problem Formulation

Initial State: The agent starts from the user's current/last-known location with a set of candidate locations where the lost item may be present.

State Representation: A state can contain the current location and the remaining candidate locations to be searched.

Action Set: Move to an adjacent/connected location, search the current location, mark a location as searched, and update the candidate set.

Successor Function: Generates the valid next locations reachable from the current location according to the map.

Goal Test: The lost object is found, or all relevant candidate locations have been searched.

Cost Metric: A basic cost can be defined using movement distance and search effort. Additional experiments may include search time and location risk.

5. Data / Environment Sourcing

The initial prototype will use a manually created/simulated campus or building grid/graph so that controlled experiments can be performed. Different map sizes, obstacle configurations, candidate locations, and search scenarios will be generated for benchmarking. If external datasets, maps, libraries, or reference material are incorporated later, they will be cited according to APA requirements.

6. Proposed Algorithmic Approaches

Approach	Role	Purpose
BFS	Baseline	Provide an uninformed-search benchmark and measure nodes expanded, time, and path/search cost.
Greedy Best-First Search	Comparison	Use heuristic information to prioritize promising locations.
A*	Advanced	Combine actual cost g(n) with heuristic estimate h(n) using f(n)=g(n)+h(n).

The main heuristic for a grid environment may use Manhattan distance when movement is restricted to horizontal and vertical directions. A probability-based priority mechanism may be added as an extension to represent the likelihood that an item is located at a candidate location.

7. Planned Tooling & Libraries

- Programming Language: Python
- Algorithms/Data Structures: heapq, collections, custom graph/grid structures
- Data Handling: CSV/Pandas where required
- Visualization: Matplotlib and/or interactive grid visualization
- Prototype Interface: Streamlit
- Development Environment: VS Code / Jupyter Notebook

8. Proposed System Workflow

1. User enters the lost object, last-known location, current location, and relevant visited locations.
2. The system converts the environment into a graph/grid-based state-space representation.
3. Candidate locations and their associated search information are initialized.
4. BFS is executed as the baseline search.
5. Greedy Best-First Search and A* are executed using heuristic information.
6. The system records path/search cost, execution time, and nodes expanded.
7. The algorithms are compared across small, medium, and large test environments.
8. The final Streamlit interface visualizes the map, search process, recommended search order, and performance results.

9. Planned Experimental Evaluation

Experiments will vary the size of the environment, number of candidate locations, obstacle density, and start/goal configurations. The following metrics will be collected:

- Execution time
- Number of generated/expanded nodes
- Total path/search cost
- Path length or movement distance
- Success/failure cases
- Memory usage where feasible

Results will be presented using comparative tables and graphs. Failure cases such as unreachable locations, blocked paths, incorrect last-known locations, and large state spaces will also be analyzed.

10. Expected Deliverables

- Formal PEAS and state-space formulation.
- Working BFS baseline implementation.
- Working Greedy Best-First and A* implementations.
- Heuristic design and analysis.
- Benchmark dataset/test-instance generator.
- Comparative performance tables and graphs.
- Interactive Streamlit prototype.
- Final technical report and GitHub repository.